

PROYECTO SECURENET

Infraestructura de Acceso Remoto Seguro y Auditoría Perimetral

Integrantes: Luis Ignacio Luquero Hernández, Susana Rascado Varela,
Jorge Garrido Medina, Sebastian Jofiel López Ospino

Documentación:

<https://drive.google.com/drive/folders/1AI5e8eRaulwJaVF0b6D0IjHPVGylteTc?usp=sharing>

Repositorio:

https://github.com/ZaitecS25-Internship-projects/I_O25_VPN_y_seguridad_perimetral/tree/main/Proyecto_Vpn_Seguridad_Perimetral

Índice

1. Introducción	1
2. Objetivos de las prácticas	1
3. Tecnologías y herramientas utilizadas	2
4. Cronograma general del proyecto	3
5. Desarrollo del proyecto	8
5.1. Arquitectura y Diseño de Red	8
5.2. Despliegue de la VPN y Gestión de Usuarios	9
5.3. Seguridad Perimetral e Integración con Docker	10
5.4. Sistema de Auditoría y Monitorización	12
6. Problemas y retos encontrados	15
6.1. Conflicto de Red entre Docker e Iptables	15
6.2. Ausencia de Logs Nativos en WireGuard	15
6.3. Identificación de Usuarios y Automatización	15
7. Aportaciones individuales	16
7.1. Luis Ignacio Luquero Hernández	16
7.2. Susana Rascado Varela	17
7.3. Jorge Garrido Medina	18
7.4. Sebastian Jofiel López Ospino	19
8. Manual de usuario	20
8.1 Acceso inicial a la Red Privada (VPN)	20
8.2 Uso de los Servicios Protegidos	20
8.3 Recepción de Reportes Automatizados	21
8.4 Configuración del Stack de Infraestructura (Docker)	21
8.5 Seguridad Perimetral: Política "Deny All" (Iptables)	21
8.6 Automatización de Auditoría y Monitorización (Bash Scripts)	22
9. Conclusiones	23
9.1 Valoración General del Proyecto	23
9.2 Aspectos Positivos y Negativos	23
9.3 Cumplimiento de Objetivos	23
9.4 Mejoras Futuras	24
10. Anexos	25
11. Presentación técnica del proyecto	27
12. Presentación comercial del proyecto	27

1. Introducción

SecureNet es una solución profesional de acceso remoto cifrado diseñada e implementada para la empresa ZAITEC. El proyecto consiste en el despliegue de una infraestructura VPN basada en el protocolo WireGuard sobre un servidor Ubuntu Linux que ya contaba con una configuración base de Docker, backups y monitorización.

La importancia de este proyecto radica en la necesidad empresarial de ofrecer conexiones externas seguras y trazables a sus sistemas internos. El objetivo principal ha sido convertir el servidor en un "gateway" o puerta de enlace protegida, de forma que los servicios críticos (como la administración de aplicaciones o la visualización de métricas) dejen de estar expuestos a Internet y solo sean accesibles a través del túnel cifrado.

Dentro de nuestro ciclo formativo de ASIR, este trabajo integra competencias clave en administración de sistemas, seguridad perimetral mediante cortafuegos y gestión de servicios en red. Hemos aprendido a configurar infraestructuras donde el servidor actúa como un muro de protección, minimizando la superficie de ataque y transformando una necesidad técnica en un servicio real, seguro y funcional para la empresa.

2. Objetivos de las prácticas

El objetivo principal de este proyecto es transformar un servidor convencional en una infraestructura de acceso restringido bajo el modelo de "Confianza Cero" (Zero Trust). Con esto, se garantiza que solo el personal autorizado de ZAITEC pueda interactuar con los servicios internos, creando un prototipo de seguridad que la empresa pueda replicar para sus clientes.

Objetivos Generales:

- Implementar una solución de conexión remota segura que garantice la confidencialidad de los datos en tránsito.
- Centralizar la gestión de accesos y la auditoría del sistema para mejorar la capacidad de respuesta ante incidentes.

- Consolidar los conocimientos adquiridos durante el ciclo formativo en un entorno de producción real y profesional.

Objetivos Específicos:

- **Seguridad Perimetral:** Configurar una política de cortafuegos "Deny All" (denegar todo por defecto) para cerrar todos los puertos del servidor al exterior, dejando únicamente operativo el puerto de la VPN.
- **Gestión de Identidades:** Crear y gestionar perfiles de usuario (Admin, Operador, Cliente) con permisos segmentados y direcciones IP fijas dentro de la red privada.
- **Monitorización Avanzada:** Desplegar un sistema de logs y métricas que permita visualizar en tiempo real quién se conecta, desde dónde y a qué servicios accede.
- **Automatización de Alertas:** Desarrollar scripts que informen automáticamente vía Telegram sobre el estado de salud del servidor y posibles intentos de intrusión.
- **Optimización de Red:** Resolver los conflictos de enrutamiento entre el motor de contenedores y el sistema operativo para asegurar que el firewall filtre correctamente todo el tráfico.

3. Tecnologías y herramientas utilizadas

Para el desarrollo de SecureNet, se ha seleccionado un conjunto de tecnologías de código abierto que destacan por su eficiencia, seguridad y gran presencia en el sector profesional actual.

Infraestructura y Red:

- **Ubuntu Server:** Sistema operativo base sobre el que se despliega toda la solución.
- **WireGuard:** Protocolo de VPN moderno, elegido por su velocidad y mayor seguridad criptográfica frente a opciones tradicionales.
- **Docker:** Motor de contenedores utilizado para el despliegue y aislamiento de los servicios. Se ha empleado Docker Compose para la orquestación y gestión unificada de todo el stack tecnológico mediante archivos de configuración YAML.

Seguridad:

- **Iptables:** Herramienta de administración del cortafuegos del kernel de Linux. Es la pieza central para filtrar el tráfico y aplicar la política de denegación total (Deny All).
- **Fail2Ban:** Sistema de prevención de intrusiones que bloquea automáticamente IPs que muestran comportamientos maliciosos o ataques de fuerza bruta.

Monitorización y Observabilidad (Stack PLG):

- **Grafana:** Plataforma de visualización para crear paneles de control interactivos y métricas en tiempo real.
- **Loki y Promtail:** Sistema de gestión de logs que permite centralizar, indexar y buscar registros de todo el sistema de forma eficiente.

Automatización y Gestión:

- **Bash Scripting:** Lenguaje utilizado para programar las tareas de auditoría y los scripts de monitorización personalizados.
- **Telegram API:** Utilizada para el envío inmediato de alertas y reportes de estado directamente a los dispositivos móviles del equipo técnico.
- **GitHub:** Plataforma de control de versiones para gestionar el código de los scripts y las configuraciones del proyecto.
- **Miro y Trello:** Herramientas fundamentales para el diseño de la arquitectura de red y la gestión de tareas mediante metodología Kanban.

4. Cronograma general del proyecto

El desarrollo de SecureNet se ha estructurado en un ciclo de seis semanas, siguiendo una metodología de trabajo incremental. Cada fase ha sido diseñada para consolidar la base técnica antes de añadir capas de complejidad y seguridad.

- **Semana 1: Planificación y Diseño de Arquitectura:**
 - Definición de requisitos con ZAITEC y elección de tecnología VPN (WireGuard).
 - Diseño del esquema de red y flujo de datos en Miro.
 - Configuración del entorno colaborativo en GitHub y carga inicial de tareas en Trello.

- **Semana 2: Despliegue de Infraestructura VPN:**
 - Instalación del motor Docker y despliegue del servicio wg-easy mediante Docker Compose.
 - Activación de IP Forwarding en el host para habilitar el enrutamiento.
 - Creación de clientes con IPs fijas y aislamiento del servicio web para acceso exclusivo vía VPN.
- **Semana 3: Seguridad Perimetral y Hardening:**
 - Implementación de la política "Deny All" y cierre de puertos al exterior.
 - Migración crítica de UFW a Iptables para garantizar que Docker no evada el cortafuegos.
 - Diseño de perfiles y reglas de acceso segmentadas (Admin, Operador, Cliente).
- **Semana 4: Monitorización y Centralización de Logs:**
 - Integración de Grafana con Loki para la visualización centralizada de registros.
 - Configuración de la retención y rotación de logs mediante logrotate.
 - Auditoría de tráfico con Wireshark y escaneos Nmap externos para validar el firewall.
- **Semana 5: Automatización y Alertas en Tiempo Real:**
 - Desarrollo de scripts en Bash para la auditoría de WireGuard y monitorización del sistema.
 - Integración con la API de Telegram para el envío automático de informes y alertas.
 - Programación de tareas automáticas mediante Cron para diagnósticos periódicos.
- **Semana 6: Validación Final y Documentación:**
 - Redacción de la memoria técnica y consolidación de evidencias del proyecto.
 - Diseño del panel final en Miro con la arquitectura definitiva.
 - Preparación de la demo final y estructura de la presentación técnica.

Evidencias de gestión (Trello):

Figura 1 (Semanas 1-2): Inicialización del proyecto y primer despliegue funcional de la VPN con WireGuard.

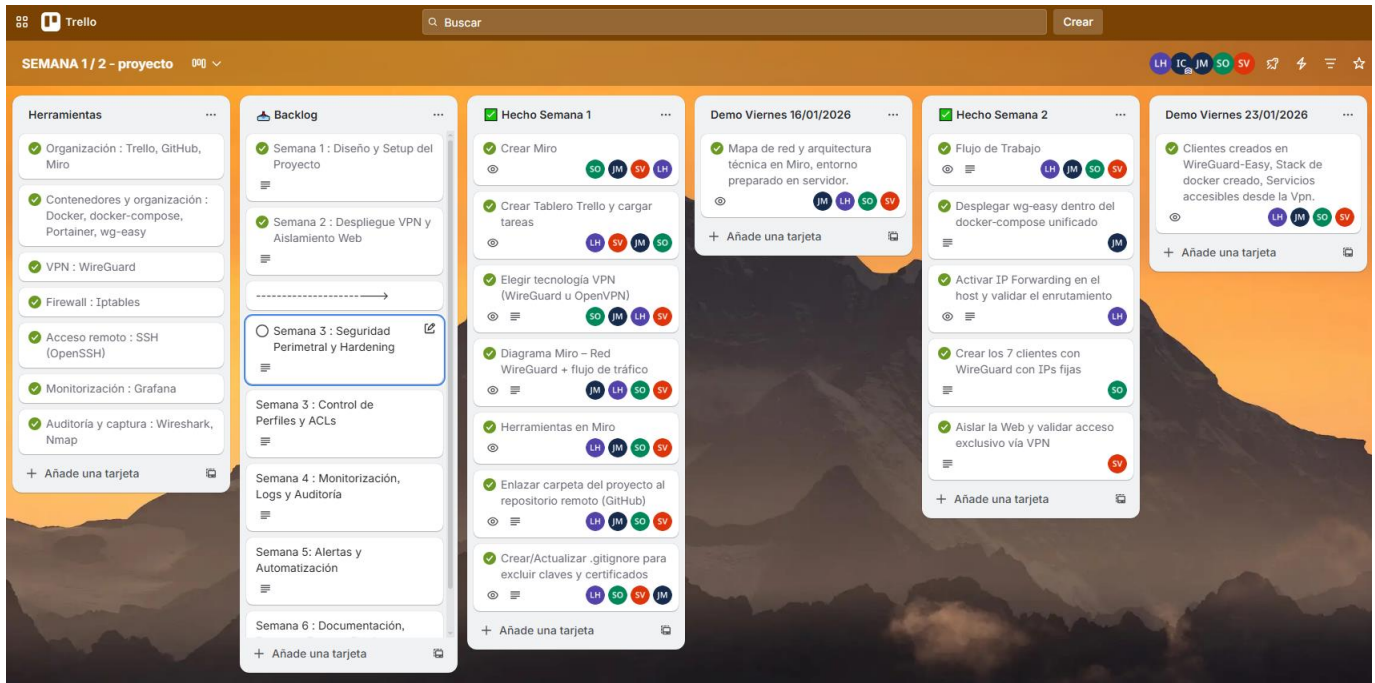


Figura 2 (Semanas 3-4): Bloque crítico de seguridad perimetral y centralización de registros para auditoría.

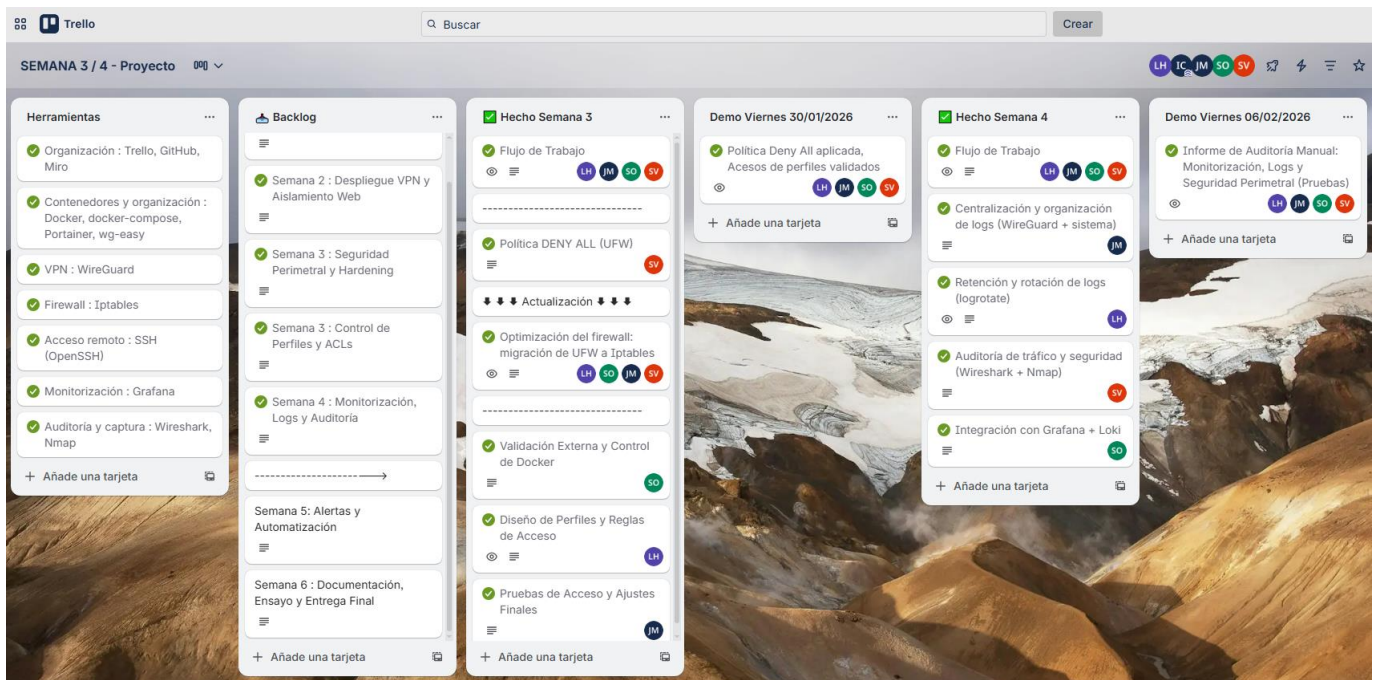
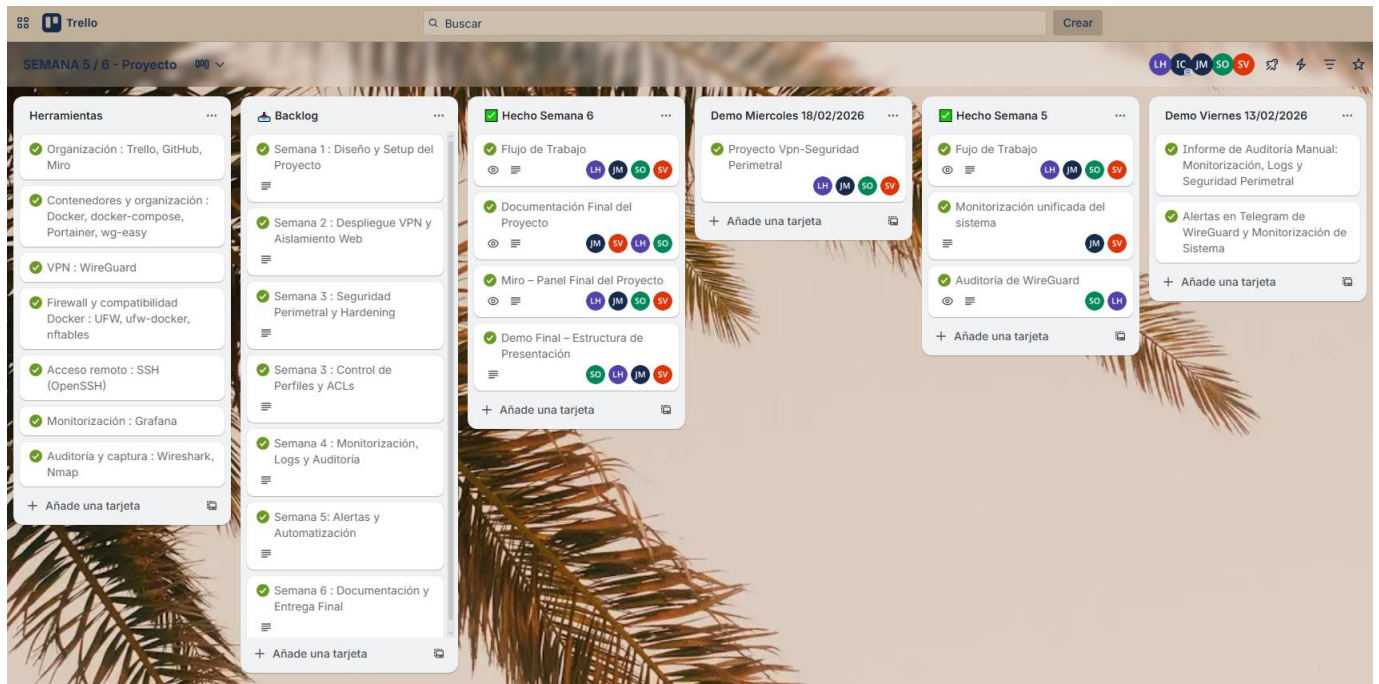
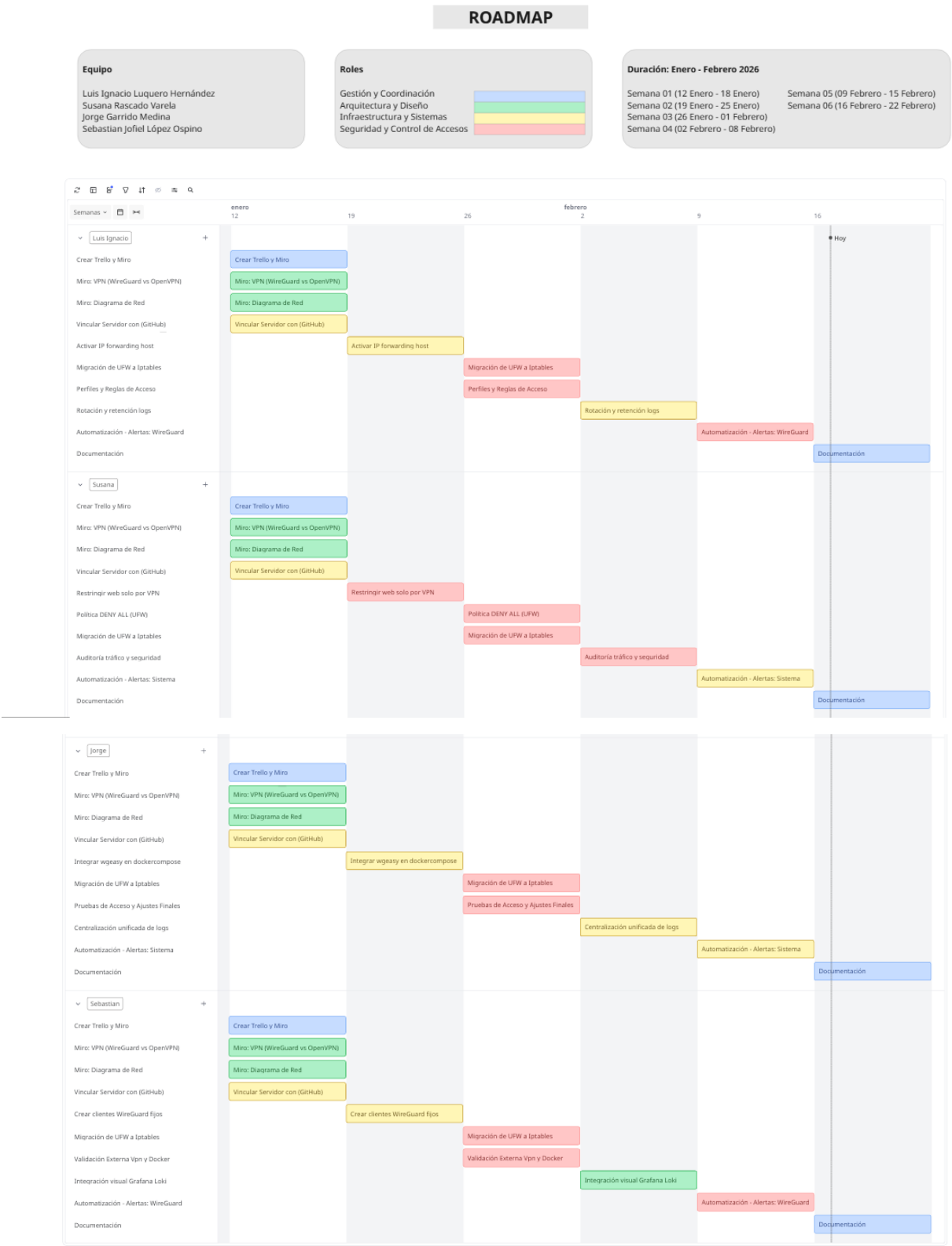


Figura 3 (Semanas 5-6): Fase de finalización con la implementación de avisos automáticos y consolidación de la documentación técnica.



Evidencias de gestión (Roadmap):

Figura 4: Roadmap General del Proyecto y Cronograma de Fases



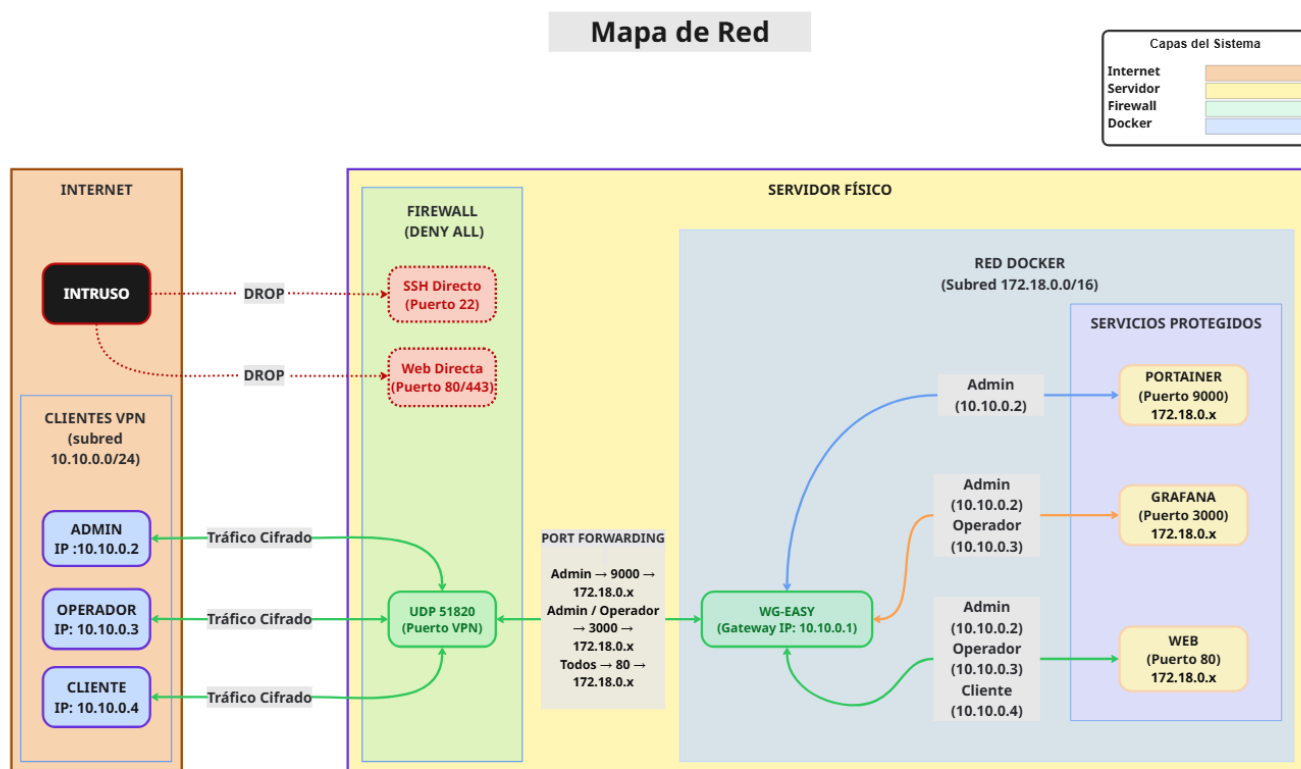
5. Desarrollo del proyecto

El desarrollo técnico de **SecureNet** se ha basado en la creación de capas de seguridad superpuestas, asegurando que el servidor no solo sea una herramienta de conexión, sino un entorno blindado.

5.1. Arquitectura y Diseño de Red

La infraestructura se diseñó bajo un esquema de **Gateway Seguro**. El tráfico externo llega al servidor a través de Internet, pero es interceptado inmediatamente por el firewall. Solo aquellos paquetes que pertenecen al protocolo WireGuard (puerto UDP 51820) tienen permiso para entrar. Una vez el usuario se autentica mediante su llave criptográfica, se le asigna una IP privada dentro del rango 10.10.0.X, permitiéndole navegar por la red interna de Docker.

Figura 5: Mapa de Red - Arquitectura de Capas y Flujo de Tráfico Cifrado.



5.2. Despliegue de la VPN y Gestión de Usuarios

Se optó por el uso de wg-easy para facilitar la gestión de los perfiles. El despliegue se realizó mediante contenedores, lo que permite un aislamiento total del servicio. Se configuraron 7 clientes con direccionamiento IP estático para poder aplicar reglas de firewall personalizadas más adelante:

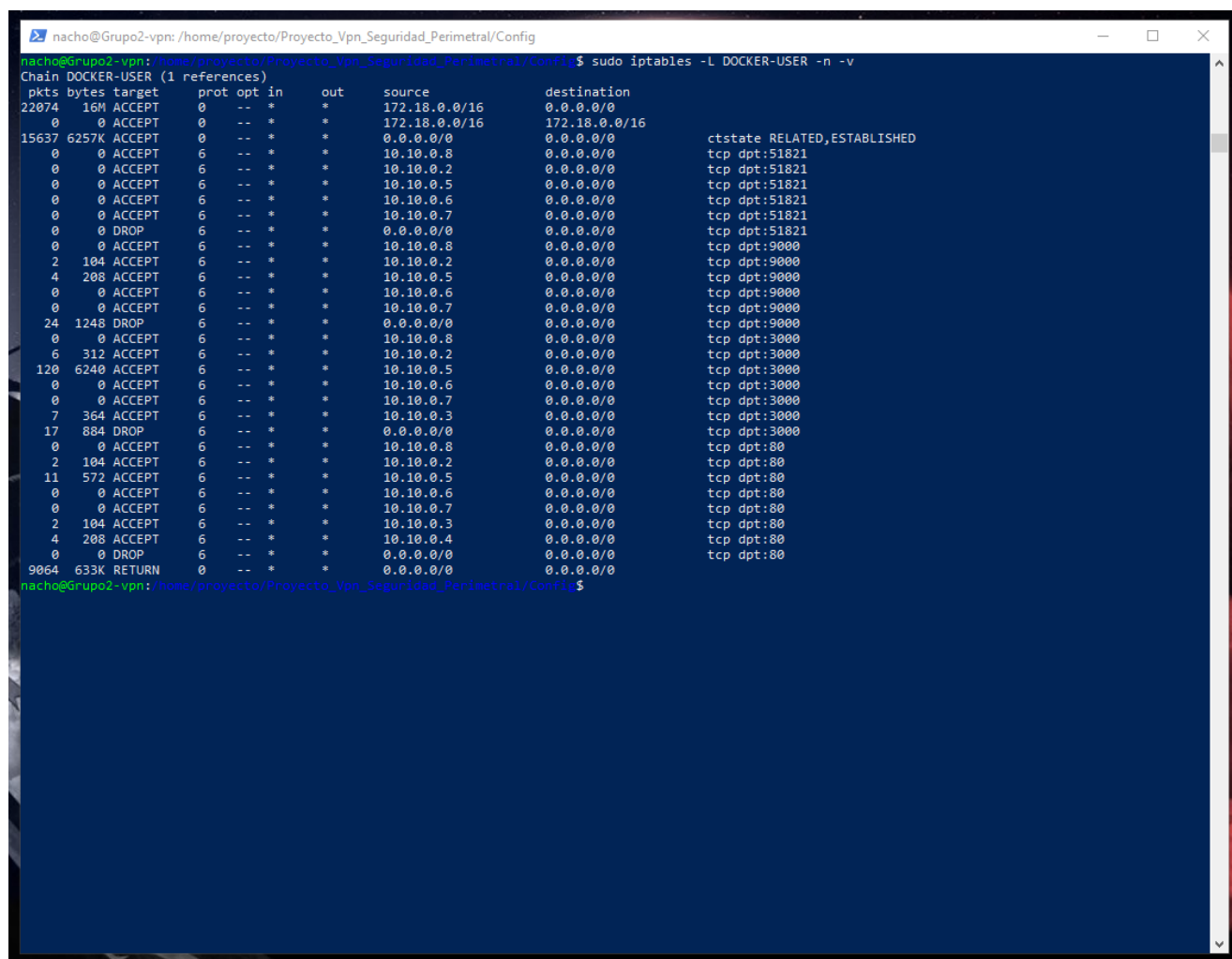
- **Admin/ Root (Luis, Jorge, Susana, Sebastian):** Acceso total a la infraestructura.
- **Operador:** Acceso a herramientas de monitorización y gestión.
- **Cliente:** Acceso restringido únicamente al servicio web final.

5.3. Seguridad Perimetral e Integración con Docker

Este fue el punto más crítico del desarrollo. Debido a que Docker gestiona sus propias reglas en iptables, se decidió eliminar UFW para evitar conflictos y tomar el control manual del cortafuegos.

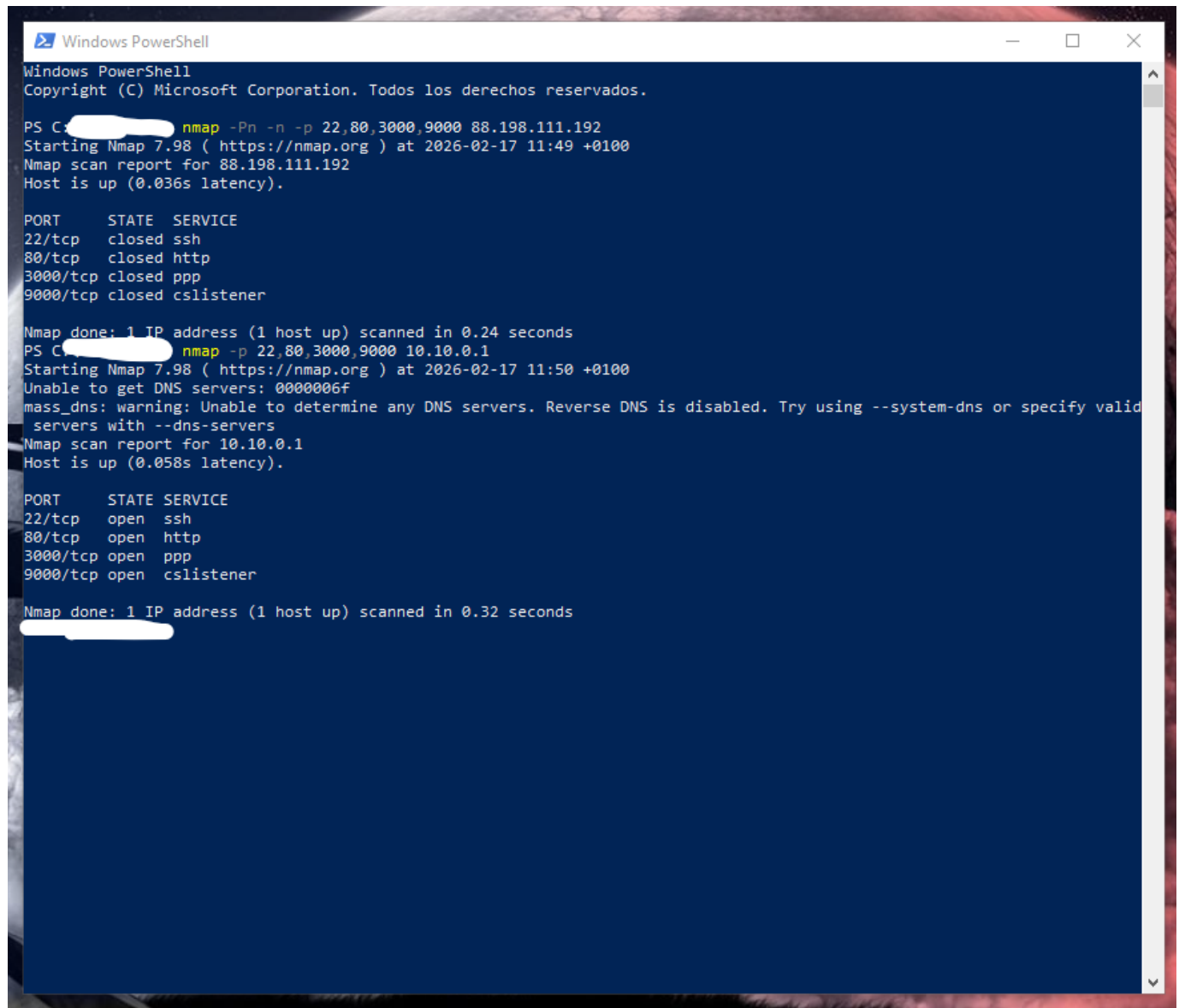
- **Política DROP:** Se configuró el servidor para que descarte cualquier paquete entrante por defecto.
- **Cadena DOCKER-USER:** Se implementaron reglas específicas en esta cadena para asegurar que, aunque Docker abra puertos internamente, estos no sean accesibles desde la IP pública del servidor, sino solo desde la red de la VPN.

Figura 6: Volcado de reglas en la cadena DOCKER-USER y efectividad del filtrado perimetral.



```
nacho@Grupo2-vpn: /home/proyecto/Proyecto_Vpn_Seguridad_Perimetral/Config
nacho@Grupo2-vpn: /home/proyecto/Proyecto_Vpn_Seguridad_Perimetral/Config$ sudo iptables -L DOCKER-USER -n -v
Chain DOCKER-USER (1 references)
pkts bytes target prot opt in out source destination
22074 16M ACCEPT 0 -- * * 172.18.0.0/16 0.0.0.0/0
0 0 ACCEPT 0 -- * * 172.18.0.0/16 172.18.0.0/16
15637 6257K ACCEPT 0 -- * * 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLISHED
0 0 ACCEPT 6 -- * * 10.10.0.8 0.0.0.0/0 tcp dpt:51821
0 0 ACCEPT 6 -- * * 10.10.0.2 0.0.0.0/0 tcp dpt:51821
0 0 ACCEPT 6 -- * * 10.10.0.5 0.0.0.0/0 tcp dpt:51821
0 0 ACCEPT 6 -- * * 10.10.0.6 0.0.0.0/0 tcp dpt:51821
0 0 ACCEPT 6 -- * * 10.10.0.7 0.0.0.0/0 tcp dpt:51821
0 0 DROP 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:51821
0 0 ACCEPT 6 -- * * 10.10.0.8 0.0.0.0/0 tcp dpt:9000
2 104 ACCEPT 6 -- * * 10.10.0.2 0.0.0.0/0 tcp dpt:9000
4 208 ACCEPT 6 -- * * 10.10.0.5 0.0.0.0/0 tcp dpt:9000
0 0 ACCEPT 6 -- * * 10.10.0.6 0.0.0.0/0 tcp dpt:9000
0 0 ACCEPT 6 -- * * 10.10.0.7 0.0.0.0/0 tcp dpt:9000
24 1248 DROP 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:9000
0 0 ACCEPT 6 -- * * 10.10.0.8 0.0.0.0/0 tcp dpt:3000
6 312 ACCEPT 6 -- * * 10.10.0.2 0.0.0.0/0 tcp dpt:3000
120 6240 ACCEPT 6 -- * * 10.10.0.5 0.0.0.0/0 tcp dpt:3000
0 0 ACCEPT 6 -- * * 10.10.0.6 0.0.0.0/0 tcp dpt:3000
0 0 ACCEPT 6 -- * * 10.10.0.7 0.0.0.0/0 tcp dpt:3000
7 364 ACCEPT 6 -- * * 10.10.0.3 0.0.0.0/0 tcp dpt:3000
17 884 DROP 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:3000
0 0 ACCEPT 6 -- * * 10.10.0.8 0.0.0.0/0 tcp dpt:80
2 104 ACCEPT 6 -- * * 10.10.0.2 0.0.0.0/0 tcp dpt:80
11 572 ACCEPT 6 -- * * 10.10.0.5 0.0.0.0/0 tcp dpt:80
0 0 ACCEPT 6 -- * * 10.10.0.6 0.0.0.0/0 tcp dpt:80
0 0 ACCEPT 6 -- * * 10.10.0.7 0.0.0.0/0 tcp dpt:80
2 104 ACCEPT 6 -- * * 10.10.0.3 0.0.0.0/0 tcp dpt:80
4 208 ACCEPT 6 -- * * 10.10.0.4 0.0.0.0/0 tcp dpt:80
0 0 DROP 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
9064 633K RETURN 0 -- * * 0.0.0.0/0 0.0.0.0/0
nacho@Grupo2-vpn: /home/proyecto/Proyecto_Vpn_Seguridad_Perimetral/Config$
```

Figura 7. Validación de invisibilidad WAN vs. acceso VPN.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

PS C:\> nmap -Pn -n -p 22,80,3000,9000 88.198.111.192
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-17 11:49 +0100
Nmap scan report for 88.198.111.192
Host is up (0.036s latency).

PORT      STATE SERVICE
22/tcp    closed ssh
80/tcp    closed http
3000/tcp  closed ppp
9000/tcp  closed cslistener

Nmap done: 1 IP address (1 host up) scanned in 0.24 seconds
PS C:\> nmap -p 22,80,3000,9000 10.10.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-17 11:50 +0100
Unable to get DNS servers: 0000006f
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid
servers with --dns-servers
Nmap scan report for 10.10.0.1
Host is up (0.058s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
3000/tcp  open  ppp
9000/tcp  open  cslistener

Nmap done: 1 IP address (1 host up) scanned in 0.32 seconds
```

Como se observa en la captura superior, se realizó un test de penetración básico con Nmap 7.98 para validar la infraestructura:

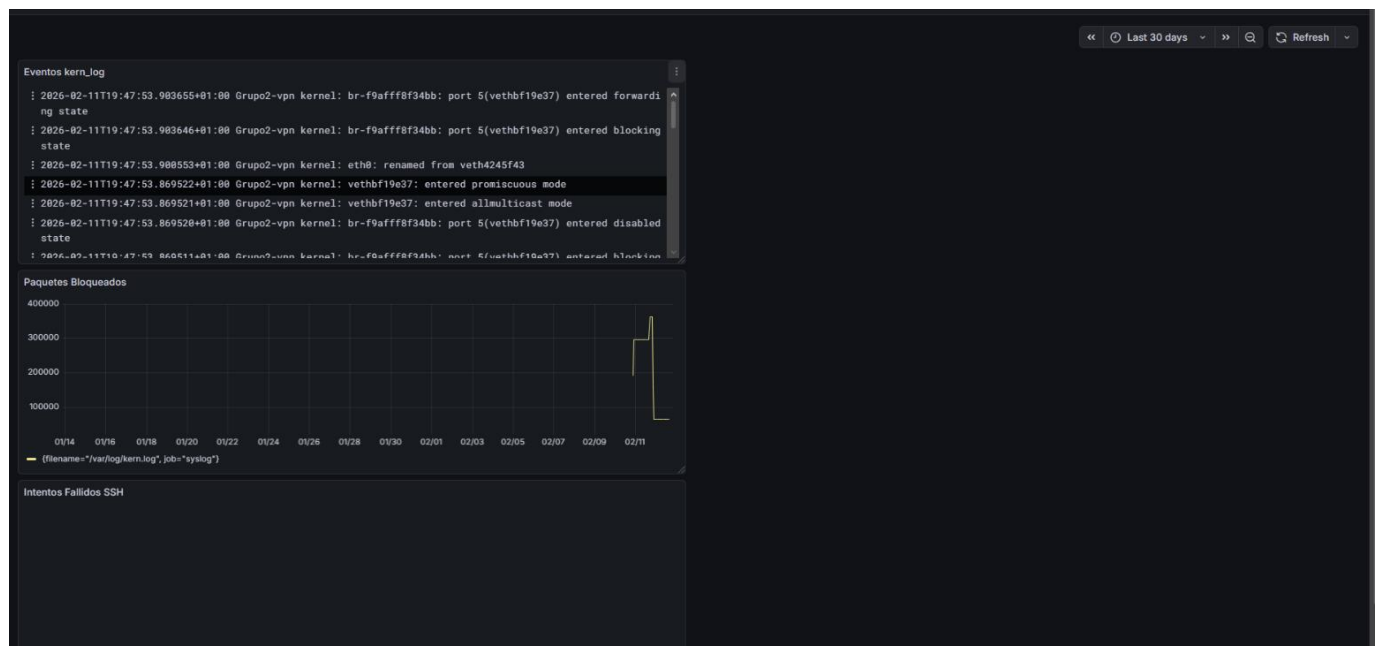
- **Bloqueo Externo (IP 88.198.111.192):** Los puertos 22, 80, 3000 y 9000 devuelven un estado closed. Esto confirma que el servidor cumple con la política "Deny All" y que Docker ya no expone servicios a internet de forma automática.
- **Acceso por Túnel (IP 10.10.0.1):** Al repetir el escaneo bajo la interfaz de la VPN, los mismos puertos aparecen como open. Esto valida que el túnel de WireGuard es el único camino autorizado para la gestión y uso de los servicios protegidos .

5.4. Sistema de Auditoría y Monitorización

Para cerrar el ciclo de desarrollo, se implementó el stack de monitorización PLG (Promtail, Loki, Grafana) junto con scripts personalizados.

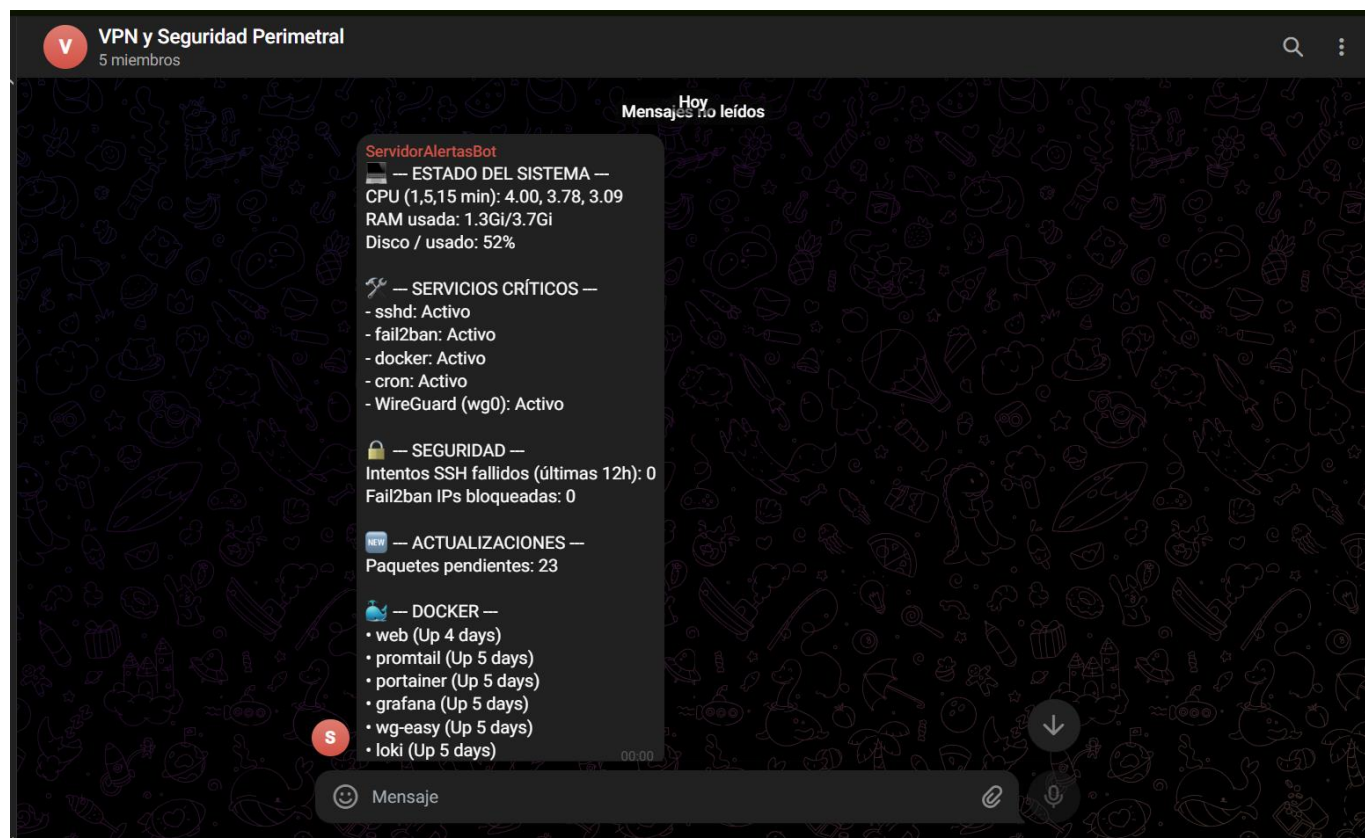
- **Monitorización de Logs:** Promtail recolecta los logs de sistema y de WireGuard, enviándolos a Loki para su indexación.

Figura 8: Panel de monitorización en Grafana: Visualización de eventos de kernel y métricas de red.



- **Automatización con Bash:** Se crearon los scripts `wireguard.sh` y `monitorizacion.sh`. Estos analizan el estado de los *handshakes* y la salud del hardware, enviando un resumen visual al equipo técnico a través de un Bot de Telegram cada 12 horas.

Figura 9-11: Recepción de alertas de seguridad y auditoría en tiempo real.



V

VPN y Seguridad Perimetral

5 miembros

🔍

⋮

AUDITORÍA DE WIREGUARD

Estado general de la VPN:

• Interfaz wg0 activa

• Peers configurados: 7

• Peers con actividad en las últimas 24h: 7

Detalle por peer:

Peer: Root

• IP: 10.10.0.2

• Estado: Desconectado ❌

• Último handshake: 2026-02-11 20:36:52

Peer: Operador

• IP: 10.10.0.3

• Estado: Desconectado ❌

• Último handshake: 2026-02-11 20:36:11

Peer: Cliente

• IP: 10.10.0.4

• Estado: Desconectado ❌

• Último handshake: 2026-02-12 17:32:30

Peer: Sebastian

• IP: 10.10.0.5

• Estado: Desconectado ❌

• Último handshake: 2026-02-13 18:04:25

Peer: Nacho

• IP: 10.10.0.6

• Estado: Conectado (últimas 12h) ✅

• Último handshake: 2026-02-16 18:39:58

S

📄

📄

Mensaje

📎

🎤

V

VPN y Seguridad Perimetral

5 miembros

🔍

⋮

• Último handshake: 2026-02-11 20:36:11 Hoy

Peer: Cliente

• IP: 10.10.0.4

• Estado: Desconectado ❌

• Último handshake: 2026-02-12 17:32:30

Peer: Sebastian

• IP: 10.10.0.5

• Estado: Desconectado ❌

• Último handshake: 2026-02-13 18:04:25

Peer: Nacho

• IP: 10.10.0.6

• Estado: Conectado (últimas 12h) ✅

• Último handshake: 2026-02-16 18:39:58

Peer: Jorge

• IP: 10.10.0.7

• Estado: Desconectado ❌

• Último handshake: 2026-02-10 20:30:51

Peer: Susana

• IP: 10.10.0.8

• Estado: Desconectado ❌

• Último handshake: 2026-02-13 16:50:43

❌ Errores técnicos del sistema en WireGuard (últimas 24h):

Sin errores técnicos detectados

S

📄

📄

Mensaje

📎

🎤

14

6. Problemas y retos encontrados

Durante el desarrollo de SecureNet, el equipo se enfrentó a diversos desafíos técnicos que requirieron una investigación profunda y la toma de decisiones críticas para asegurar la infraestructura.

6.1. Conflicto de Red entre Docker e Iptables

Este fue el hito técnico más complejo del proyecto. Descubrimos que Docker manipula directamente las reglas de iptables para permitir el tráfico a los contenedores, creando "atajos" que ignoran las configuraciones de cortafuegos más sencillas como UFW.

- **Solución:** Decidimos desinstalar UFW para evitar conflictos y tomamos el control manual del cortafuegos. Esto nos permitió insertar nuestras reglas de bloqueo antes de que Docker procese el tráfico, garantizando que los servicios internos solo respondan si la petición viene desde dentro del túnel VPN.

6.2. Ausencia de Logs Nativos en WireGuard

WireGuard es un protocolo extremadamente ligero que, por diseño, no genera archivos de log de conexiones (registros de entrada y salida). Esto suponía un problema para la trazabilidad y la auditoría detallada que requería la empresa.

- **Solución:** Para solucionar esta carencia, configuramos el sistema para capturar la actividad a través de los logs del kernel y del sistema (syslog y kern.log). Implementamos Promtail para monitorizar estos archivos del host en tiempo real y configuramos logrotate con compresión automática para asegurar que el almacenamiento no se sature, manteniendo un histórico de evidencias de 4 semanas.

6.3. Identificación de Usuarios y Automatización

Al no existir un registro de nombres de usuario dentro de los comandos de WireGuard (solo se visualizan claves públicas e IPs), la monitorización manual resultaba ineficiente para el equipo técnico.

- **Solución:** Desarrollamos un script de auditoría avanzado (wireguard.sh) que utiliza una base de alias asociados a IPs fijas. El script consulta el estado de la interfaz, traduce las IPs a los alias correspondientes de cada perfil y calcula el tiempo transcurrido desde el último *handshake*. Esta información se procesa y se envía automáticamente a Telegram, convirtiendo datos técnicos del sistema en un informe de auditoría legible y estructurado.

7. Aportaciones individuales

El desarrollo de SecureNet se ha llevado a cabo de forma conjunta por los cuatro integrantes del equipo. Aunque para la organización del proyecto en Trello se asignaron responsables a ciertas tareas técnicas, todas las fases (desde el diseño en Miro hasta la automatización con scripts) han sido validadas, discutidas y ejecutadas en colaboración. La comunicación constante ha permitido que las soluciones técnicas, como la migración a Iptables o la estructura del bot de alertas, sean fruto del trabajo en equipo.

7.1. Luis Ignacio Luquero Hernández

- **Semana 1:** Creación del tablero Trello, diseño de arquitectura en Miro, elección de tecnología VPN (WireGuard) e inicialización del repositorio en GitHub con su respectivo gitignore.
- **Semana 2:** Activación técnica del **IP Forwarding** en `/etc/sysctl.conf`, aplicación con `sysctl -p` y validación del enrutamiento hacia la red interna de Docker.
- **Semana 3: Optimización del firewall mediante la migración de UFW a Iptables** e implementación de la política "deny all" para asegurar que el tráfico de Docker sea filtrado en la cadena **DOCKER-USER**.
- **Semana 3: Diseño de perfiles de acceso:** Creación de la matriz de asignación de IPs manual para la segmentación técnica de los perfiles Admin, Operador y Cliente.

- **Semana 4: Generación del Informe de Auditoría:** Redacción conjunta del informe integral de monitorización, detallando el estado de estabilidad, riesgos detectados y conclusiones de seguridad del sistema.
- **Semana 4: Rotación y Retención de Logs:** Configuración de la política de rotación semanal, retención de 4 ciclos y compresión diferida en logrotate para asegurar la integridad de /var/log/syslog y auth.log.
- **Semana 5: Integración con la API de Telegram** para el sistema de avisos automáticos y programación de la ejecución cíclica mediante **Cron** cada 12 horas.
- **Semana 6:** Redacción final de la memoria técnica y preparación del material visual para la demostración del proyecto.

7.2. Susana Rascado Varela

- **Semana 1:** Creación del tablero Trello, diseño de arquitectura en Miro, elección de tecnología VPN (WireGuard) e inicialización del repositorio en GitHub con su respectivo gitignore.
- **Semana 2: Aislamiento del servicio web:** Eliminación del mapeo de puertos públicos en el archivo docker-compose.yml para garantizar el acceso exclusivo a través del túnel VPN.
- **Semana 3: Optimización del firewall mediante la migración de UFW a Iptables** e implementación de la política "deny all" para asegurar que el tráfico de Docker sea filtrado en la cadena **DOCKER-USER**.
- **Semana 3: Hardening de red:** Ejecución técnica de las reglas de filtrado y limpieza de las configuraciones residuales de UFW en el servidor.
- **Semana 4: Generación del Informe de Auditoría:** Redacción conjunta del informe integral de monitorización, detallando el estado de estabilidad, riesgos detectados y conclusiones de seguridad del sistema.

- **Semana 4: Validación mediante Capturas y Nmap:** Realización de capturas Wireshark para verificar el cifrado UDP 51820 y ejecución de escaneos Nmap externos para certificar la política de mínima exposición.
- **Semana 5: Monitorización de salud del sistema:** Desarrollo de las funciones de captura de métricas de hardware (CPU, RAM y almacenamiento) y estado de servicios críticos.
- **Semana 6:** Redacción final de la memoria técnica y preparación del material visual para la demostración del proyecto.

7.3. Jorge Garrido Medina

- **Semana 1:** Creación del tablero Trello, diseño de arquitectura en Miro, elección de tecnología VPN (WireGuard) e inicialización del repositorio en GitHub con su respectivo gitignore.
- **Semana 2: Instalación y configuración de la VPN:** Despliegue del contenedor wg-easy mediante Docker y validación del acceso interno al panel de gestión administrativa.
- **Semana 3: Optimización del firewall mediante la migración de UFW a Iptables** e implementación de la política "deny all" para asegurar que el tráfico de Docker sea filtrado en la cadena **DOCKER-USER**.
- **Semana 3: Pruebas de Acceso y Ajustes Finales:** Validación real de las reglas desde cada perfil de usuario y ajuste de las ACLs según el comportamiento de red esperado.
- **Semana 4: Generación del Informe de Auditoría:** Redacción conjunta del informe integral de monitorización, detallando el estado de estabilidad, riesgos detectados y conclusiones de seguridad del sistema.
- **Semana 4: Auditoría de Accesos e Iptables:** Extracción de handshakes activos mediante wg show y análisis de intentos de acceso bloqueados registrados en iptables.log.

- **Semana 5: Monitorización de salud del sistema:** Desarrollo de las funciones de captura de métricas de hardware (CPU, RAM y almacenamiento) y estado de servicios críticos.
- **Semana 6:** Redacción final de la memoria técnica y preparación del material visual para la demostración del proyecto.

7.4. Sebastian Jofiel López Ospino

- **Semana 1:** Creación del tablero Trello, diseño de arquitectura en Miro, elección de tecnología VPN (WireGuard) e inicialización del repositorio en GitHub con su respectivo gitignore.
- **Semana 2: Gestión técnica de cuentas de usuario:** Generación de perfiles de acceso, archivos de configuración .conf y códigos QR para los clientes de la VPN.
- **Semana 3: Optimización del firewall mediante la migración de UFW a Iptables** e implementación de la política "deny all" para asegurar que el tráfico de Docker sea filtrado en la cadena DOCKER-USER.
- **Semana 3: Auditoría Externa con Nmap:** Ejecución de escaneos desde una red externa para certificar el bloqueo efectivo de puertos y la invisibilidad del servidor frente a ataques.
- **Semana 4: Generación del Informe de Auditoría:** Redacción conjunta del informe integral de monitorización, detallando el estado de estabilidad, riesgos detectados y conclusiones de seguridad del sistema.
- **Semana 4: Métricas de Auditoría y Observabilidad:** Definición e interpretación de métricas en kern.log y auth.log, e implementación de paneles en Grafana para la visualización de seguridad.
- **Semana 5: Integración con la API de Telegram** para el sistema de avisos automáticos y programación de la ejecución cíclica mediante Cron cada 12 horas.
- **Semana 6:** Redacción final de la memoria técnica y preparación del material visual para la demostración del proyecto.

8. Manual de usuario

Este apartado describe los pasos necesarios para que un usuario autorizado acceda a la infraestructura y utilice los servicios protegidos, detallando a su vez la arquitectura técnica que garantiza que dichos servicios permanezcan aislados y seguros bajo una política de Deny All.

8.1 Acceso inicial a la Red Privada (VPN)

El acceso a todo el ecosistema está restringido. Para entrar, el usuario debe seguir estos pasos:

1. **Instalación del Cliente:** Descargar e instalar el software oficial de WireGuard en su dispositivo (Windows, Linux, macOS, Android o iOS).
2. **Importación del Perfil:** El administrador le proporcionará un archivo .conf o un código QR generado por el panel wg-easy.
3. **Activación del Túnel:** Una vez importado, el usuario debe pulsar en "Activar". Si el estado cambia a "Activo" y hay intercambio de datos, el usuario ya forma parte de la red segura 10.10.0.x.

8.2 Uso de los Servicios Protegidos

Una vez conectado a la VPN, el usuario puede acceder a los servicios mediante las siguientes URLs internas (que no funcionan desde el internet abierto):

- **Página Web del Proyecto:** Acceso a la web corporativa o de servicios en <http://10.10.0.1>.
- **Panel de Gestión de Contenedores (Admin):** Acceso a Portainer en <http://10.10.0.1:9000>.
- **Dashboard de Monitorización (Admin/Operador):** Visualización de métricas en Grafana a través de <http://10.10.0.1:3000>.

8.3 Recepción de Reportes Automatizados

Los usuarios con perfil de administrador/auditor estarán integrados en el Canal de Telegram.

De forma automática, recibirán:

- Un reporte de salud del servidor cada 12 horas (CPU, RAM, ataques bloqueados).
- Un reporte de auditoría de conexiones detallando qué usuarios (Peers) están conectados y quiénes llevan tiempo inactivos.

8.4 Configuración del Stack de Infraestructura (Docker)

El despliegue se basa en una arquitectura de microservicios orquestada mediante Docker Compose, utilizando una red bridge con un MTU de 1350 para optimizar el encapsulamiento de datos dentro del túnel VPN.

Se ha implementado el IP Binding (amarre de interfaces) como medida primaria de aislamiento:

- **Aislamiento de Servicios:** Los servicios críticos (Nginx, Portainer, Grafana, Loki) están vinculados específicamente a la IP interna 10.10.0.1. Esto garantiza que las aplicaciones no escuchen en la IP pública, siendo invisibles para cualquier usuario externo a la VPN.
- **Puerta de Enlace Única:** El contenedor wg-easy es el único que expone un puerto al exterior (51820/udp), actuando como el único punto de entrada cifrado.
- **Persistencia de Datos:** Se utilizan volúmenes locales para asegurar que la configuración de WireGuard y las bases de datos de métricas e identidades permanezcan íntegras ante reinicios del sistema.

8.5 Seguridad Perimetral: Política "Deny All" (Iptables)

El proyecto soluciona el bypass de seguridad que Docker realiza sobre UFW mediante la implementación manual de reglas en la cadena DOCKER-USER. Esto permite un filtrado estricto antes de que el tráfico sea encaminado a los contenedores.

Análisis de Reglas y ACLs

Basándonos en el volcado real del sistema (iptables -L -n), se aplica una jerarquía de "Denegar por Defecto":

1. **Permisos Específicos:** Se habilitan reglas ACCEPT para permitir el tráfico a los puertos 80, 9000, 3100 y 3000 únicamente si el origen es una de las IPs autorizadas de la VPN (10.10.0.2 a 10.10.0.8).
2. **Cierre Total (DROP):** Cualquier paquete dirigido a estos servicios desde el exterior es descartado automáticamente. El firewall registra estos intentos, permitiendo auditar el "ruido" de ataques bloqueados.

8.6 Automatización de Auditoría y Monitorización (Bash Scripts)

El sistema cuenta con una capa de inteligencia operativa mediante scripts en Bash integrados con la API de Telegram, proporcionando observabilidad total sin intervención manual.

Monitorización del Sistema (monitorizacion.sh)

Script encargado de recolectar la salud del host y enviarla cada 12 horas vía **Cron**:

- **Hardware:** Carga de CPU, uso de RAM y ocupación de disco mediante procesamiento con awk.
- **Servicios:** Verificación activa de sshd, fail2ban, docker y cron.
- **Seguridad:** Reporte de intentos fallidos de acceso SSH y conteo de bloqueos activos.

Auditoría de Identidad VPN (wireguard.sh)

Script especializado en la capa VPN que gestiona la trazabilidad de los usuarios:

- **Mapeo de Alias:** Traduce las claves criptográficas a nombres reales (Nacho, Susana, Jorge, Sebastian, etc.).
- **Análisis de Handshake:** Determina el estado de conexión basándose en el tiempo transcurrido desde el último intercambio de claves.
- **Depuración:** Escanea el journalctl en busca de errores técnicos específicos de la interfaz wg0.

9. Conclusiones

9.1 Valoración General del Proyecto

El desarrollo de SecureNet ha cumplido satisfactoriamente con todos los objetivos planteados al inicio de las prácticas. Se ha logrado transformar un servidor expuesto en una infraestructura blindada bajo el paradigma de "Confianza Cero" (Zero Trust).

- **Eficacia de la Solución:** La implementación de WireGuard ha demostrado ser una elección acertada por su bajo impacto en los recursos y su alta velocidad, cumpliendo con la necesidad de ZAITEC de ofrecer un servicio ágil y seguro.
- **Éxito en la Seguridad Perimetral:** El hito más relevante ha sido el control total del cortafuegos a través de Iptables. Resolver el conflicto entre Docker y el firewall nos ha permitido garantizar que los servicios críticos sean totalmente invisibles desde el exterior, cumpliendo la política de Deny All.

9.2 Aspectos Positivos y Negativos

- **Positivo:** La integración de la API de Telegram ha aportado un valor diferencial, permitiendo una administración proactiva y móvil. Además, la migración a Iptables nos ha dado un conocimiento profundo sobre el funcionamiento del kernel de Linux y la red de contenedores.
- **Negativo/Reto:** La mayor dificultad fue la carencia de logs nativos en WireGuard, lo que nos obligó a investigar y desarrollar scripts personalizados para obtener una trazabilidad real de los usuarios.

9.3 Cumplimiento de Objetivos

Se han alcanzado los objetivos iniciales al 100%:

1. **Conexión cifrada:** Operativa y estable.
2. **Control de acceso:** Segmentado por IPs fijas y perfiles.
3. **Monitorización:** Desplegada mediante el stack PLG y alertas en tiempo real.

4. **Aislamiento:** Verificado mediante escaneos Nmap externos que confirman el cierre de puertos.

9.4 Mejoras Futuras

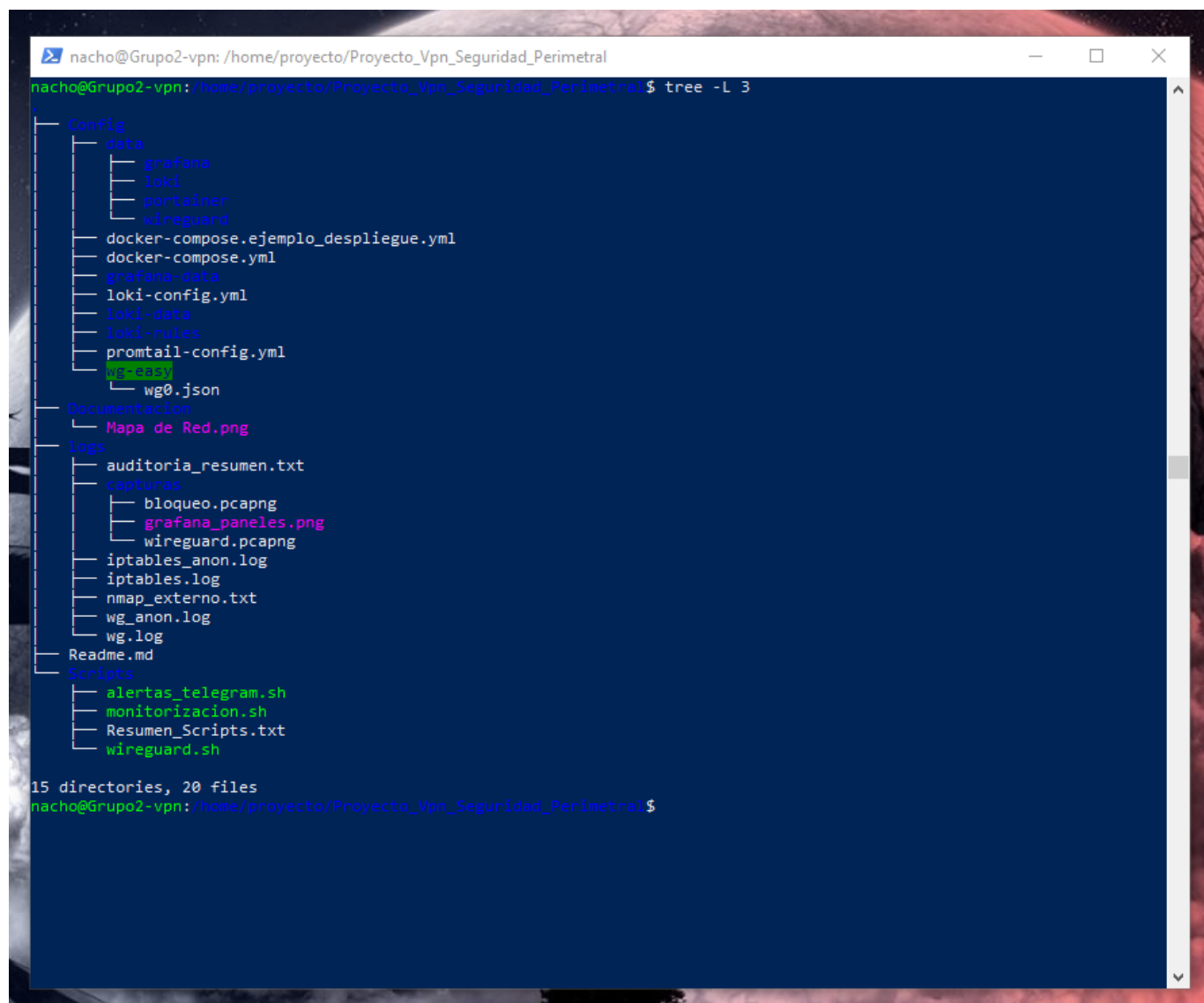
Para escalar este prototipo comercial de ZAITEC, se proponen las siguientes mejoras:

- **Alta Disponibilidad:** Configurar un segundo nodo VPN en clúster.
- **Doble Factor de Autenticación (2FA):** Añadir una capa de seguridad extra antes del túnel.
- **Dashboard Histórico:** Crear paneles de Grafana más avanzados para análisis de tráfico a largo plazo.

10. Anexos

Anexo I: Organización del Repositorio y Entorno de Trabajo

Figura 12: Estructura jerárquica de directorios del proyecto.

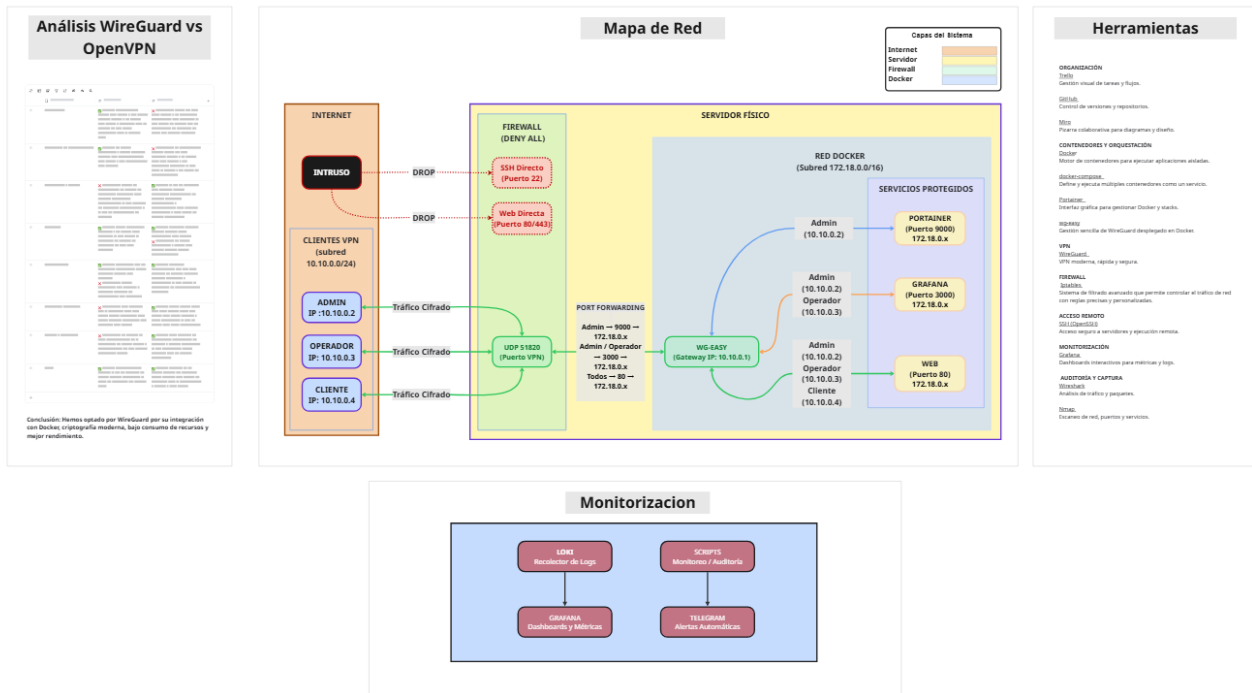


```
nacho@Grupo2-vpn: /home/proyecto/Proyecto_Vpn_Seguridad_Perimetral
nacho@Grupo2-vpn:/home/proyecto/Proyecto_Vpn_Seguridad_Perimetral$ tree -L 3
.
├── Config
│   ├── data
│   │   ├── grafana
│   │   ├── loki
│   │   ├── portainer
│   │   └── wireguard
│   ├── docker-compose.ejemplo_despliegue.yml
│   ├── docker-compose.yml
│   ├── grafana-data
│   ├── loki-config.yml
│   ├── loki-data
│   ├── loki-rules
│   ├── promtail-config.yml
│   └── wireguard
│       └── wg0.json
├── Documentacion
│   └── Mapa de Red.png
├── logs
│   ├── auditoria_resumen.txt
│   ├── capturas
│   │   ├── bloqueo.pcapng
│   │   ├── grafana_paneles.png
│   │   └── wireguard.pcapng
│   ├── iptables_anon.log
│   ├── iptables.log
│   ├── nmap_externo.txt
│   ├── wg_anon.log
│   └── wg.log
├── Readme.md
└── Scripts
    ├── alertas_telegram.sh
    ├── monitorizacion.sh
    ├── Resumen_Scripts.txt
    └── wireguard.sh

15 directories, 20 files
nacho@Grupo2-vpn:/home/proyecto/Proyecto_Vpn_Seguridad_Perimetral$
```

Anexo II: Diseño Conceptual

Figura 13: Lienzo de Diseño y Stack Tecnológico.



Anexo III: Código Fuente y Scripts de Automatización

Con el objetivo de facilitar la revisión técnica, el mantenimiento y el control de versiones, todo el código desarrollado para SecureNet se encuentra centralizado en el repositorio oficial del proyecto.

Se puede acceder a la última versión estable de los scripts de monitorización, ejemplos de orquestación de contenedores..., en el siguiente enlace:

https://github.com/ZaitecS25-Internship-projects/I_O25_VPN_y_seguridad_perimetral/tree/main/Proyecto_Vpn_Seguridad_Perimetral

11. Presentación técnica del proyecto

Se adjunta a continuación el enlace a la demostración técnica funcional de la arquitectura SecureNet:

https://drive.google.com/file/d/1l8NC5j6DQnAyLRsUbef7wZkiKM2sg_LG/view?usp=drive_link

12. Presentación comercial del proyecto

A continuación, se facilita el recurso audiovisual correspondiente a la propuesta de valor y visión comercial del producto para ZAITEC:

https://drive.google.com/file/d/1oMRP2Rw__QD5431FmWZ8uEa43A9gfDXS/view?usp=drive_link